



Python Programming Fundamentals

Scope and Pure Functions

Programming Fundamentals Team

SETU

2026-06-18



Outline

1. Local vs Global Scope
2. The LEGB Rule
3. Avoiding Global Variables
4. Pure/Impure Functions
5. Constants and Module-Level Names
6. Intermediate Variables



Outline

- 1. Local vs Global Scope**
2. The LEGB Rule
3. Avoiding Global Variables
4. Pure/Impure Functions
5. Constants and Module-Level Names
6. Intermediate Variables



1. Local vs Global Scope

Variables live in a **scope** — the region of code where they are accessible.

```
message = "I am global"    # global scope

def show_message():
    local_msg = "I am local" # local scope
    print(message)           # can read global
    print(local_msg)         # can read local

show_message()
print(message)              # OK
# print(local_msg) # NameError – doesn't exist here
```



1. Local vs Global Scope

Variables live in a **scope** — the region of code where they are accessible.

```
message = "I am global"      # global scope

def show_message():
    local_msg = "I am local"  # local scope
    print(message)            # can read global
    print(local_msg)          # can read local

show_message()
print(message)                # OK
# print(local_msg)           # NameError – doesn't exist here
```

Each function call creates its own **local namespace**. When the function returns, its locals are discarded.



Outline

1. Local vs Global Scope
- 2. The LEGB Rule**
3. Avoiding Global Variables
4. Pure/Impure Functions
5. Constants and Module-Level Names
6. Intermediate Variables



2. The LEGB Rule

Python searches for names in this order:

Letter	Scope	Example
L	Local — inside current function	x defined in my_func
E	Enclosing — outer function (closures)	Nested functions
G	Global — module level	PI = 3.14159
B	Built-in — Python's built-ins	len, print, range



2. The LEGB Rule

```
x = "global"

def outer():
    x = "enclosing"
    def inner():
        # x = "local"           # uncomment to use local
        print(x)               # finds "enclosing" (E before G)
    inner()

outer()    # enclosing
```




Outline

1. Local vs Global Scope
2. The LEGB Rule
- 3. Avoiding Global Variables**
4. Pure/Impure Functions
5. Constants and Module-Level Names
6. Intermediate Variables



3. Avoiding Global Variables

Reading a global is fine. **Modifying** a global from inside a function is a **code smell**.

```
# Bad pattern – hidden global state
total = 0

def add_to_total(value):
    global total
    total += value          # modifies global – hard to test

add_to_total(10)
add_to_total(25)
print(total)              # 35
```



3. Avoiding Global Variables

Better pattern — pass data in, return data out:

```
def add_values(current_total, value):  
    return current_total + value  
  
total = 0  
total = add_values(total, 10)  
total = add_values(total, 25)  
print(total)      # 35
```

Now the function is **testable** and has **no side effects**.



Outline

1. Local vs Global Scope
2. The LEGB Rule
3. Avoiding Global Variables
- 4. Pure/Impure Functions**
5. Constants and Module-Level Names
6. Intermediate Variables



4. Pure/Impure Functions

A **pure function**:

1. Always produces the same output for the same input
2. Has **no side effects** (doesn't modify external state, no I/O)

```
# Pure – same input always gives same output, no side effects
```

```
def multiply(a, b):  
    return a * b
```

```
def clamp(value, minimum, maximum):  
    """Constrain value to [minimum, maximum] range."""  
    return max(minimum, min(maximum, value))
```

```
print(clamp(150, 0, 100))    # 100  
print(clamp(-5, 0, 100))    # 0  
print(clamp(50, 0, 100))    # 50
```



4. Pure/Impure Functions

Impure functions — have side effect/s (printing, reading input, modifying external state):

```
def get_score():          # impure – depends on user input
    return float(input("Score: "))

def log_result(msg):      # impure – side effect (printing)
    print(msg)
```

NOTE: We need both pure and impure functions. It is important to separate them. For instance a pure function `clamp` can be used in impure functions `log_result` and `get_score`. It is good practice to write pure functions first and then impure functions.



Outline

1. Local vs Global Scope
2. The LEGB Rule
3. Avoiding Global Variables
4. Pure/Impure Functions
- 5. Constants and Module-Level Names**
6. Intermediate Variables



5. Constants and Module-Level Names

Constants are module-level names that never change. By convention, use ALL_CAPS.

```
PI = 3.14159265
GRAVITY = 9.81          # m/s2
TAX_RATE = 0.23
MAX_RETRIES = 3

def circle_area(radius):
    return PI * radius ** 2

def circle_circumference(radius):
    return 2 * PI * radius

print(f"Area of r=5 circle: {circle_area(5):.2f}")
print(f"Circumference: {circle_circumference(5):.2f}")
```




5. Constants and Module-Level Names

Constants are fine at module level — they are **not expected to change** and are clearly marked.



Outline

1. Local vs Global Scope
2. The LEGB Rule
3. Avoiding Global Variables
4. Pure/Impure Functions
5. Constants and Module-Level Names
- 6. Intermediate Variables**



6. Intermediate Variables

An **intermediate variable** holds a computed result temporarily — a named stepping stone between an input and a final output.



6. Intermediate Variables

An **intermediate variable** holds a computed result temporarily — a named stepping stone between an input and a final output.

```
# Without intermediate variables – hard to read and debug
def circle_stats(r):
    return 3.14159 * r ** 2, 2 * 3.14159 * r

# With intermediate variables – clear and debuggable
def circle_stats(r):
    pi = 3.14159
    area = pi * r ** 2
    circumference = 2 * pi * r
    return area, circumference
```



6. Intermediate Variables

Benefits:

- **Readability** — the name says what the value means
- **Debugging** — you can `print()` any step to inspect it
- **Reuse** — compute once, use multiple times (e.g. `pi` above)
- **Scope** — they are local to the function, discarded when it returns



6. Intermediate Variables

When to use them:

```
# Good – complex sub-expression deserves a name
def monthly_payment(principal, annual_rate, years):
    monthly_rate = annual_rate / 12
    n_payments   = years * 12
    factor       = (1 + monthly_rate) ** n_payments
    return principal * monthly_rate * factor / (factor - 1)
```



6. Intermediate Variables

When NOT to bother:

```
# Unnecessary – wrapping something already obvious adds noise
def square(n):
    result = n * n    # pointless intermediate variable
    return result

# Just write:
def square(n):
    return n * n
```



6. Intermediate Variables

Rule of thumb: if naming the value makes the code clearer or avoids computing it twice, use an intermediate variable. If it is a one-use wrapper around something already obvious, skip it.



Thanks for Watching - Any questions?

